

INCOIS 3D Ocean Data Visualization

Hackathon Interview Q&A Preparation

Problem Statement: SIH26067

General & Architecture Questions

1. What is the core problem your application solves?

Answer: Traditional oceanographic data (like Temperature, Salinity, and Currents) is usually viewed in flat 2D maps or complex scientific software. Our application provides a browser-native, fully interactive 4D (3D + Time) visualization platform. It allows researchers and operational oceanographers to effortlessly overlay real-world autonomous observations (like Argo floats and AMSRE satellites) directly on top of massive numerical prediction models (like Mercator) to quickly assess ocean conditions and model accuracy.

2. Can you walk us through your Technology Stack?

Answer:

- **Frontend:** Built with React and TypeScript. We use **Three.js (React Three Fiber)** for the localized 3D volumetric rendering (like depth slices and isosurfaces) and **CesiumJS** for the global geospatial mapping.
- **Backend:** Powered by **FastAPI** (Python) for extremely fast, asynchronous API endpoints.
- **Data Pipeline:** We use **Xarray** and **Zarr** to chunk and compress massive NetCDF ocean datasets. We use **Pandas** to ingest CSV-based satellite and float data.

3. How did you implement the visualization of Real vs. Model data?

Answer: We implemented a "Compare" module that acts as a real-time validation tool. The system fetches ground-truth data from actual ocean sensors. It then queries the numerical model at that exact latitude, longitude, and depth. We calculate the Root Mean Square Error (RMSE) and Bias, and plot both profiles side-by-side using Recharts, giving scientists immediate feedback on how accurate the prediction model is.

Deep Technical Questions

4. Which specific numerical prediction models are you using, and what are their specifications?

Answer: We are primarily using the **Mercator Ocean Global Reanalysis (GLORYS12V1)** and the **NOAA Sea Surface Temperature** models. The Mercator model is a state-of-the-art global eddy-resolving model with a 1/12° horizontal resolution and 50 standard vertical levels. It provides comprehensive multivariate outputs including potential temperature (`thetao`), salinity (`so`), and horizontal currents (`uo` , `vo`).

5. How do you handle the massive size of these global models in the browser without crashing it?

Answer: We never send the entire dataset to the client. The backend converts traditional NetCDF files into the **Zarr format**, which chunks the data spatially. We specifically employ a chunking strategy that optimizes horizontal spatial dimensions (e.g., lat/lon chunks of size 200x200). When the user requests a depth layer, FastAPI uses Xarray to lazily extract only that specific 2D slice directly from disk without loading the entire volume into RAM, and sends it as a highly compressed, flat JSON array.

6. Can you explain the algorithm used in your WebGL Shaders to render the volumetric Depth Slice?

Answer: When the 1D JSON array arrives at the frontend, we don't render millions of individual points. Instead, we pack the data array into a WebGL **DataTexture**. We create a PlaneGeometry, and in our custom

GLSL vertex shader, we physically displace the mesh vertices vertically based on the data to create a 3D topographic feel. In the fragment shader, we sample the DataTexture using precise UV coordinates and mathematically interpolate the values through a custom scientific colormap (like Turbo or Viridis), doing all the heavy lifting on the GPU.

7. How are you dealing with the coordinate projection differences between Cesium and your localized 3D scenes?

Answer: CesiumJS uses Earth-Centered, Earth-Fixed (ECEF) coordinates, which are excellent for global mapping but terrible for localized 3D precision (due to 64-bit floating point limitations in WebGL). When a user clicks a point on the Cesium globe to enter the workstation, we transition to Three.js and create a custom Cartesian local tangent plane. This centers the local coordinate system `(0,0,0)` exactly on the observation coordinates, eliminating floating-point jitter and allowing millimeter-level precision in our volumetric shaders.

8. How exactly are the RMSE and Bias metrics calculated in the Compare module?

Answer: When a user selects an Argo float, we extract its vertical depth profile. The backend then performs a nearest-neighbor spatial interpolation (`xarray.Dataset.sel(method="nearest")`) on the 3D Mercator grid to find the model's prediction at the exact Latitude and Longitude of the float. We then align the depth arrays.

- **Bias** is calculated as the mean difference across all depth levels, indicating if the model is systematically over-predicting or under-predicting.
- **RMSE** is calculated as the square root of the mean of the squared differences, providing a standard metric for the absolute error magnitude.

9. What was the most significant performance bottleneck you encountered, and how was it solved?

Answer: Rendering 500,000+ points of AMSRE satellite data globally brought Cesium to a halt. We overcame this with a hybrid approach:

1. For interaction, we used Pandas on the backend to deterministically sample exactly 50 valid points to act as clickable observation markers.
2. For the visual representation, instead of rendering individual points, we instructed Cesium to load a `SingleTileImageryProvider`. We generate a full 2D surface image from the Zarr model on-the-fly and wrap it around the globe as a transparent heatmap layer, maintaining a flawless 60 FPS while visualizing the entire Earth.